
Indizes

BITLC | Tom Selig
11. August 2026

Grundlagen

- Indizes sind eine Möglichkeit, die Leistung (Performance) einer Datenbank zu optimieren, da sie eine erhebliche Verbesserung der Abfrageleistung ermöglichen.
- SQL-Server unterscheidet zwischen gruppierten (clustered) und nicht gruppierten (non-clustered) Indizes.

gruppierter Index

Die Daten werden nach den Indexwerten sortiert in doppelt verketteten Listen gespeichert.

nicht gruppierter Index

Zusätzliche Datenstruktur, die parallel zu den Daten angelegt wird.

Warum Indizes?

- Stellen Sie sich vor, Sie wollen eine bestimmte Information in einem Buch finden.
- Sie haben grundsätzlich zwei Möglichkeiten:
 1. Sie blättern das gesamte Buch von vorne bis hinten durch, bis Sie die Information gefunden haben.
 2. Wenn das Buch über einen Index verfügt, können Sie dort gezielt nach einem passenden Schlüsselwort suchen, und dann direkt die Stelle(n) im Buch aufschlagen.
- Die zweite Möglichkeit sollte deutlich schneller sein. Und genau so arbeiten auch Indizes in Datenbanken.

Tabellen ohne Index

- Angenommen es gibt eine Tabelle mit Mitarbeitern ohne Index:

Personal_Nr	Vorname	Nachname	Geburtsdatum	...
66-1-H-02	Hans	Hilbert	02.01.1966	...
70-1-K-01	Peter	Krause	07.08.1970	...
52-1-S-01	Karsten	Siebert	27.05.1952	...
...	...	...	...	...
75-2-A-05	Alfred	Abraham	20.09.1975	...
70-2-Z-05	Fred	Zywalski	10.06.1970	...
50-3-N-02	Martin	Niebold	25.03.1950	...
...	...	...	...	...
63-2-L-01	Anne	Leville	20.04.1963	...
63-2-L-02	Axel	Linnert	12.05.1963	...
60-2-J-01	Doris	Jablonski	02.10.1960	...

Tabellen ohne Index

- Die Daten werden in Form eines Heaps (unsortiert) gespeichert.
- SQL Server verwendet als kleinste Einheit 8 KB große Seiten.
- Mögliche Anordnung der Daten:

Seite 25

66-1-H-02	...	Hilbert	...
70-1-K-01	...	Krause	...
52-1-S-01	...	Siebert	...
...	...	...	...

Seite 26

75-2-A-05	...	Abraham	...
70-2-Z-05	...	Zywalski	...
50-3-N-02	...	Niebold	...
...	...	...	...

Seite 27

63-2-L-01	...	Leville	...
63-2-L-02	...	Linnert	...
60-2-J-01	...	Jablonski	...
...	...	...	...

Datenseiten (Heap)

- Wenn hier nach einer bestimmten Personal-Nr gesucht wird, muss die komplette Tabelle (alle Seiten) gescannt werden.

nicht gruppierter Index (non-clustered index)

nicht gruppierter Index

- Wird nun ein nicht gruppierter Index für die Spalte `Personal_Nr` erstellt, wird eine zusätzliche, zweispaltige Datenstruktur angelegt:

In der ersten Spalte werden alle Einträge der Spalte `Personal_Nr` aus der Originaltabelle **sortiert** angelegt.

In der zweiten Spalte wird dazu dann ein Zeiger erstellt, der auf den Speicherort des passenden Datensatzes verweist. Der Zeiger besteht aus:

- Datei-ID: SQL-Server-interne ID der Datendatei
- Seitennummer: Die Seite innerhalb der Datendatei
- Position: Die Position innerhalb der Seite

Beispiel:

Zeiger: 302603 => ID: 30, Seite: 26, Position: 3

nicht gruppierter Index

50-3-N-02	302603
52-1-S-01	302503
60-2-J-01	302703
63-2-L-01	302701
63-2-L-02	302702
66-1-H-02	302501
70-1-K-01	302502
70-2-Z-05	302602
75-2-A-05	302601
...	...

Indexstruktur

Seite 25

66-1-H-02	...	Hilbert	...
70-1-K-01	...	Krause	...
52-1-S-01	...	Siebert	...
...	...	...	...

Seite 26

75-2-A-05	...	Abraham	...
70-2-Z-05	...	Zywalski	...
50-3-N-02	...	Niebold	...
...	...	...	...

Seite 27

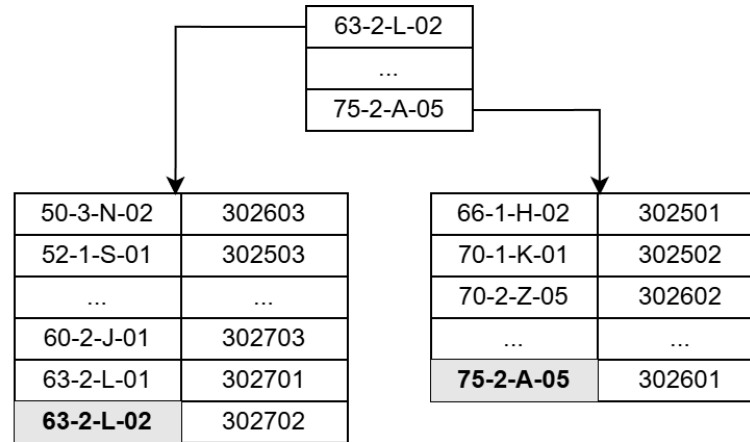
63-2-L-01	...	Leville	...
63-2-L-02	...	Linnert	...
60-2-J-01	...	Jablonski	...
...	...	...	...

Datenseiten (Heap)

nicht gruppierter Index

- Auch der Index kann über mehrere Seiten verteilt sein (8 KB!).
- Der SQL Server legt dann eine als B-Tree (Balanced Tree) bezeichnete Struktur an.
- Dabei handelt es sich um eine auf dem Kopf stehende Baum-Struktur. Der oberste Knoten ist die Wurzel, die Knoten der untersten Ebene sind die Blätter, dazwischen liegen die Äste.
- Die Wurzel und alle Ast-Ebenen enthalten Verweise auf die jeweils tieferliegende Ebene (den größten Wert der nächsten Ebene).
- Die Blattebene enthält die eigentlichen Index-Seiten, auf denen der Verweis zu den Originaldaten gespeichert ist.

nicht gruppierter Index



Indexstruktur

Seite 25

66-1-H-02	...	Hilbert	...
70-1-K-01	...	Krause	...
52-1-S-01	...	Siebert	...
...	...	...	...

Seite 26

75-2-A-05	...	Abraham	...
70-2-Z-05	...	Zywalski	...
50-3-N-02	...	Niebold	...
...	...	...	...

Seite 27

63-2-L-01	...	Leville	...
63-2-L-02	...	Linnert	...
60-2-J-01	...	Jablonski	...
...	...	...	...

Datenseiten (Heap)

nicht gruppierter Index

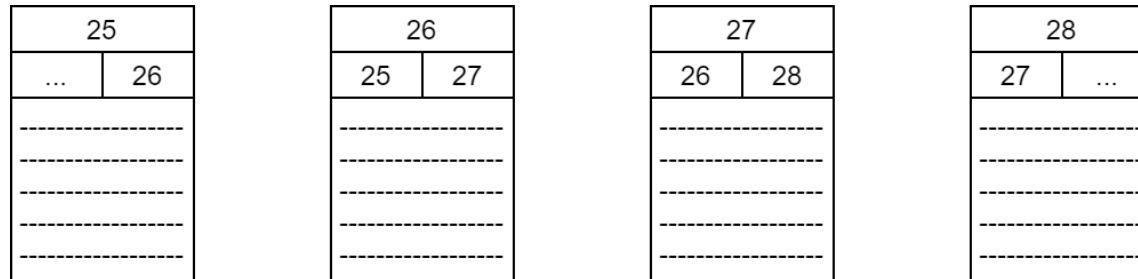
- Vorgehensweise bei der Suche von 70-1-K-01:
 - Start ist auf der obersten Ebene, der Wurzel. Von oben nach unten wird ein Eintrag gesucht, der größer oder gleich dem Suchwert ist.
 - Dies trifft auf den letzten Wert (75-2-A-05) zu, daher folgt SQL Server diesem Verweis zur nächsten, in diesem Fall rechten, Index-Seite.
 - Die nächste Index-Seite wird wieder von oben nach unten durchsucht, bis ein Wert größer oder gleich dem Suchwert gefunden wird.
 - Dies ist beim zweiten Eintrag der Fall. Der nächste Wert wird auch noch überprüft (falls Dubletten vorhanden), passt hier aber nicht.
 - SQL Server folgt dem Zeiger auf Seite 25, zweite Position.
- Bei mehr als zwei Index-Ebenen funktioniert das Verfahren analog.

gruppiertes Index

(clustered index)

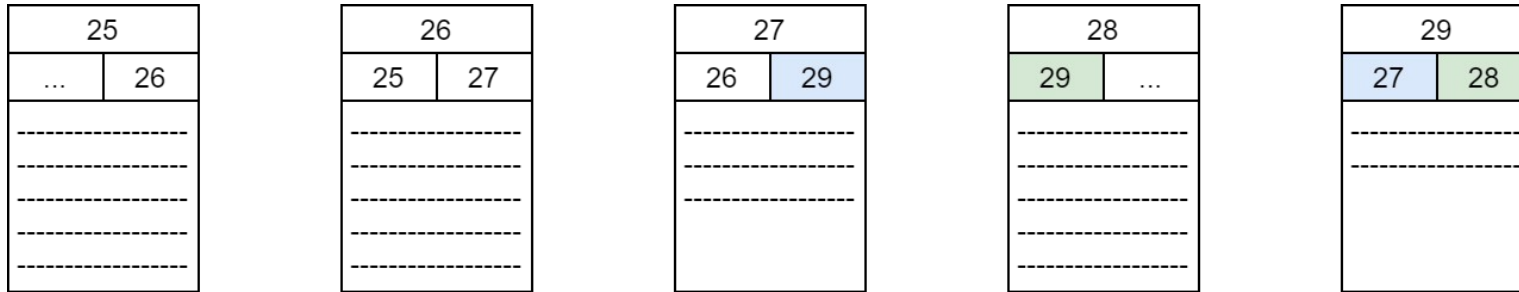
doppelt verkettete Listen

- Der gruppierte Index greift tief in die originale Datenstruktur ein.
- Die Daten werden hier nicht mehr als Heap, sondern als sortierte, doppelt verkettete Liste gespeichert.
- Auch hier gibt es wieder die 8 KB großen Seiten.
- Jede Seite der Liste enthält Verweise auf die jeweilige Vorgänger- und Nachfolger-Seite.



doppelt verkettete Listen

- Der Vorteil einer doppelt verketteten Liste ist, dass man relativ einfach Daten (Seiten) einfügen, löschen oder umsortieren kann.
- Soll z.B. zwischen die Seiten 27 und 28 eine neue Seite eingefügt werden, wird diese einfach am Ende der Liste angehängt und die Verweise für Vorgänger und Nachfolger werden geändert.



- Die Löschung oder Umsortierung einer Seite erfolgt analog.

gruppierter Index

- Da der gruppierte Index die Tabelle nach den Indexwerten physisch sortiert, kann es pro Tabelle nur einen gruppierten Index geben.
- Da die Daten für die Erstellung eines gruppierten Index ggf. umsortiert werden müssen, sollte man den gruppierten Index möglichst direkt beim Erstellen der Tabelle anlegen. Sonst kann die Tabelle bzw. Datenbank sehr lange blockiert sein.

Seite 25			
...		26	
50-3-N-02	...	Niebold	...
...	...	...	...
52-1-S-01	...	Siebert	...
58-1-M-01	...	Müller	...

Seite 26			
25		27	
60-2-J-01	...	Jablonski	...
...	...	...	...
63-2-L-01	...	Leville	...
63-2-L-02	...	Linnert	...

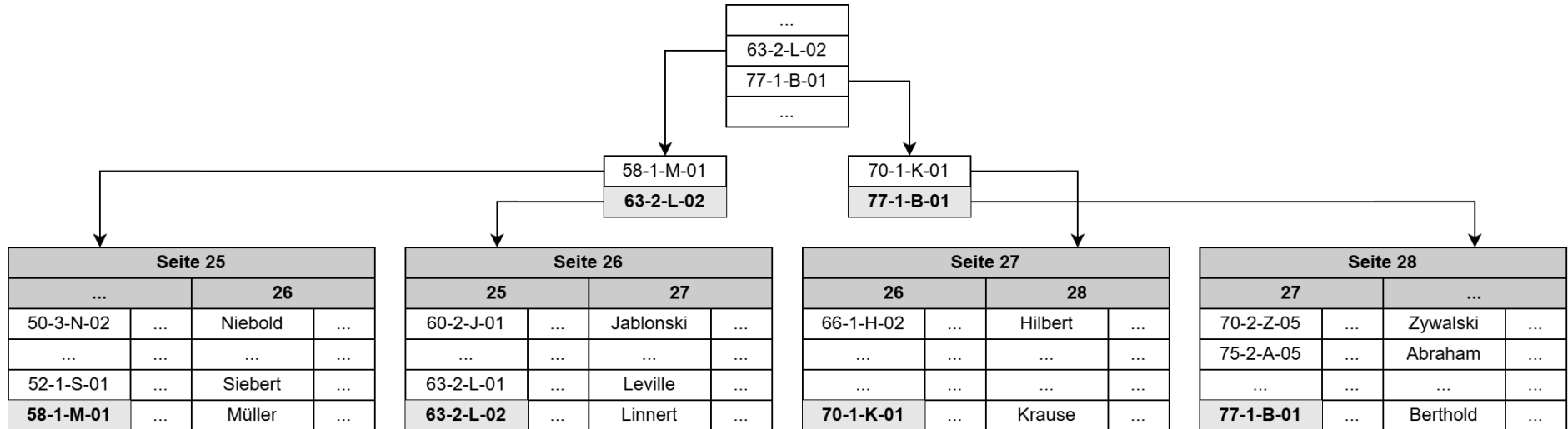
Seite 27			
26		28	
66-1-H-02	...	Hilbert	...
68-1-W-01	...	Wilhelm	...
...	...	...	...
70-1-K-01	...	Krause	...

Seite 28			
27		...	
70-2-Z-05	...	Zywalski	...
75-2-A-05	...	Abraham	...
...	...	...	...
77-1-B-01	...	Berthold	...

Datenseiten (doppelt verkettete Liste)

gruppiertter Index

- Zusätzlich zur Sortierung wird wieder eine B-Tree-Struktur zum schnellen Zugriff erstellt.
- Die Blätter sind jetzt aber die Seiten der doppelt verketteten Liste.



gruppiertter Index

gruppierter Index

- Vorgehensweise bei der Suche von 68-1-W-01:

Start auf der obersten Ebene der B-Tree-Struktur. Von oben nach unten wird ein Eintrag gesucht, der größer oder gleich dem Suchwert ist.

Dies ist beim Eintrag 77-1-B-01 der Fall. Es geht also auf der rechten Seite weiter.

Die rechte Index-Seite wird wieder von oben nach unten durchsucht, bis ein Wert größer oder gleich dem Suchwert gefunden wird.

Dies ist beim ersten Eintrag der Fall (70-1-K-01). Der Eintrag verweist auf die Seite 27.

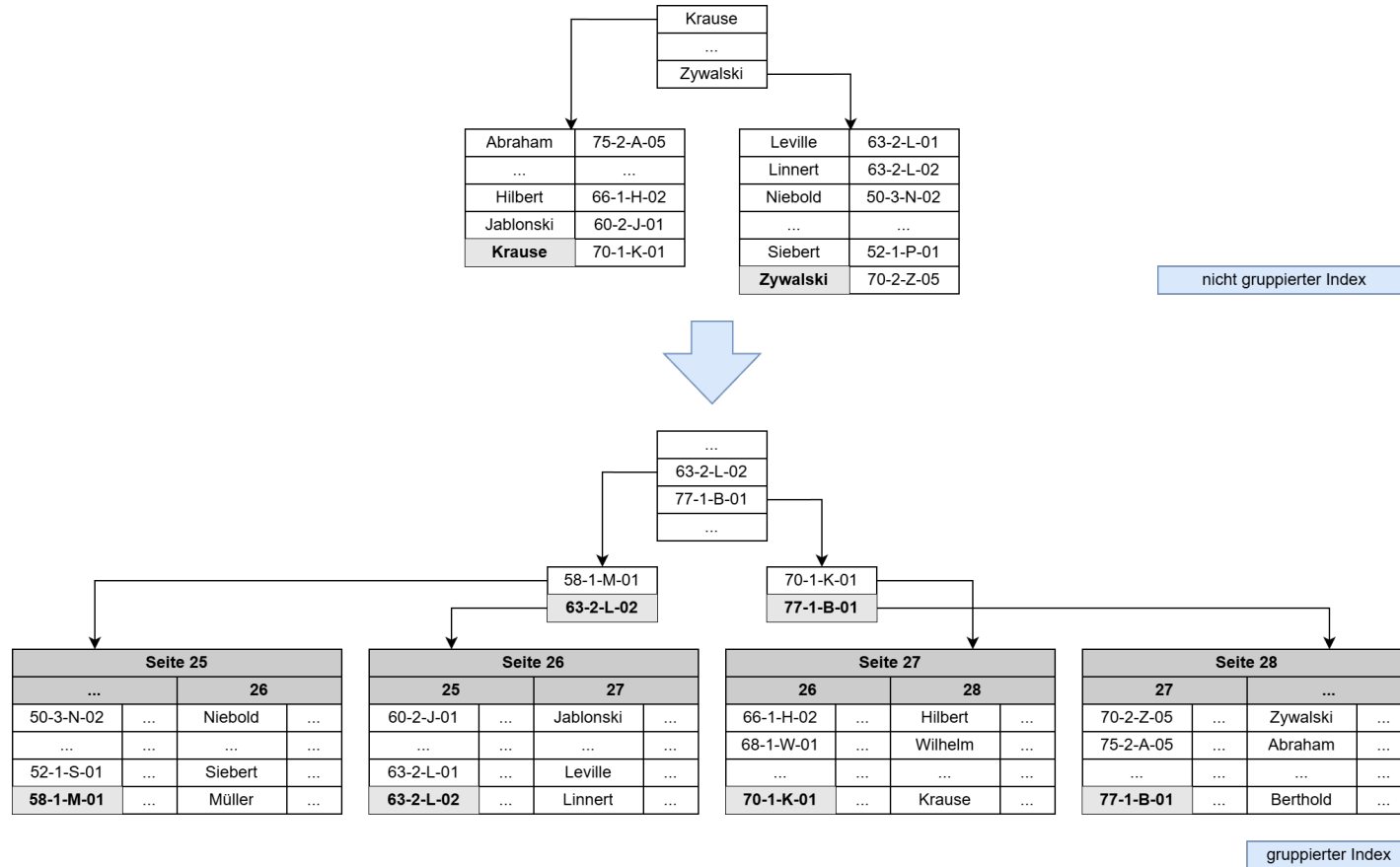
Die Seite 27 kann jetzt schnell durchsucht werden, da die Daten sortiert vorliegen, z.B. mit einer Binärsuche.

nicht gruppierter Index
auf einem gruppierten Index

nicht gruppiert auf gruppiert

- Wenn zu einer Tabelle bereits ein gruppierter Index existiert, können maximal 999 weitere, nicht gruppierte Indizes erzeugt werden.
- Bei einem nicht gruppierten Index auf einem Heap enthalten die Blatt-Seiten der B-Tree-Struktur Verweise auf den Speicherort des Datensatzes.
- Bei einem nicht gruppierten Index auf einem gruppierten Index enthalten die Blatt-Seiten Verweise auf den gruppierten Index.
- Der Zugriff ist vergleichsweise etwas langsamer, da bei jeder Abfrage immer zwei Indizes durchlaufen werden müssen.

nicht gruppiert auf gruppiert



Erstellung von Indizes

Erstellung von Indizes

- Bei der Erzeugung eines **PRIMARY KEY** wird vom SQL Server immer auch ein Index erstellt.
 - Sofern noch nicht vorhanden, wird ein gruppierter Index erstellt, d.h. die Tabelle wird nach dem Schlüsselkriterium des **PRIMARY KEY** sortiert.
 - Sollte bereits ein gruppierter Index bestehen, wird ein zusätzlicher nicht gruppierter Index für den **PRIMARY KEY** erstellt.
 - Das Verhalten lässt sich aber auch steuern, indem nach **PRIMARY KEY** entweder **CLUSTERED** oder **NONCLUSTERED** angegeben wird.
- Wird eine **UNIQUE**-Einschränkung für eine Spalte definiert, wird immer auch ein **UNIQUE**-Index für die Spalte erzeugt.

Erstellung von Indizes

- Indizes können natürlich auch manuell angelegt werden, das geschieht über den DDL-Befehl **CREATE INDEX**:

Syntax: **CREATE** [**CLUSTERED**|**NONCLUSTERED**] **INDEX** *<name>*
ON *<tabelle>* (*<spalte(n)>*);

Beispiel: **CREATE NONCLUSTERED INDEX** idx_nachname
ON Mitarbeiter (nachname);

- Die Option **NONCLUSTERED** kann hier auch weggelassen werden, da es sich um den Default-Fall handelt.
- Es können auch mehrere Spalten in der Klammer angegeben werden, dann als Liste mit Komma getrennt.

Eindeutige Indizes

- Als weitere Option kann bei der Erstellung **UNIQUE** angegeben werden. Der Index lässt dann keine doppelten Einträge zu.

Beispiel: **CREATE UNIQUE NONCLUSTERED INDEX** idx_login
ON Mitarbeiter (login);

- Ein **UNIQUE**-Index kann nicht nachträglich auf einer Spalte erstellt werden, die bereits doppelte Einträge enthält.
- **NULL** ist im **UNIQUE**-Index zwar erlaubt, es darf aber natürlich nur maximal einen **NULL**-Eintrag geben.
- Wird beim **CREATE** / **ALTER TABLE** eine **UNIQUE**-Einschränkung für eine Spalte definiert, wird auch ein **UNIQUE**-Index erzeugt.

Gefilterte Indizes

- Ein zusätzlicher nicht gruppierter Index auf einer Spalte kann die Performance von Abfragen stark verbessern.
- Allerdings kostet ein zusätzlicher Index immer Speicherplatz und ein wenig Performance für die Pflege.
- Wenn in einer Spalte nicht alle Einträge für die Suche relevant sind, kann es sinnvoll sein, einen gefilterten Index zu verwenden.
- Dies ist insbesondere dann sinnvoll, wenn es in der Spalte viele **NULL**-Einträge gibt.
- Dadurch reduziert sich der Platzbedarf und der Pflegeaufwand.

Gefilterte Indizes

- Ein gefilterter Index wird erzeugt, indem eine **WHERE**-Klausel an den **CREATE INDEX**-Befehl angehängt wird.

Beispiel: **CREATE NONCLUSTERED INDEX** idx_stadt
ON Mitarbeiter (stadt)
WHERE stadt **IS NOT NULL**;

- Ein gefilterter Index kann nur ein nicht gruppierter Index sein.
- Die **WHERE**-Bedingung muss sich nicht allein auf die indizierte Spalte beschränken.
- Der Operator **LIKE** wird von gefilterten Indizes nicht unterstützt.

Eingeschlossene Spalten

- Bei Verwendung eines nicht gruppierten Index wird dieser zunächst zur Identifizierung der relevanten Datensätze genutzt.
- Auf der Blatt-Ebene der Index-Seiten steht der Index-Wert sowie ein Verweis auf den Speicherort des Datensatzes. Weitere Informationen werden dann in einem zweiten Schritt aus den Datenseiten gelesen.
- Soll dieser Vorgang beschleunigt bzw. abgekürzt werden, können weitere Spalten in den Index eingefügt werden. Alle Informationen können dann direkt aus dem Index gelesen werden, ein Zugriff auf die Datenseiten kann entfallen.

Eingeschlossene Spalten

- Weitere, nicht indizierte Spalten können dem Index mit dem Zusatz **INCLUDE** hinzugefügt werden.

Beispiel: `CREATE NONCLUSTERED INDEX idx_stadt_incl
ON Mitarbeiter (stadt)
INCLUDE (vorname, nachname);`

- Das folgende Statement kann jetzt alle benötigten Information direkt aus dem Index lesen. Ein Zugriff auf die Datenseiten der Tabelle ist nicht mehr notwendig:

Beispiel: `SELECT vorname, nachname
FROM Mitarbeiter
WHERE stadt = 'München';`